

ДИСЦИПЛИНА	Технологии разработки серверных приложений
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям
ПРЕПОДАВАТЕЛЬ	Дворецкий Артур Геннадьевич
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Ссылка на материал:

[https://github.com/dv0retsky/fastapi-](https://github.com/dv0retsky/fastapi-tutorial/blob/main/FAPI3_Processing%20requests/FAPI3_Processing%20requests.md)

[tutorial/blob/main/FAPI3_Processing%20requests/FAPI3_Processing%20requests.md](https://github.com/dv0retsky/fastapi-tutorial/blob/main/FAPI3_Processing%20requests/FAPI3_Processing%20requests.md)

Практическое занятие №3: Обработка HTTP-запросов и их проверка

На данном занятии мы рассмотрим, как **FastAPI** обрабатывает **HTTP-запросы** и выполняет проверку данных для обеспечения целостности входящих данных. Понимание обработки запросов и проверки имеет решающее значение для создания надежных и безопасных **API-интерфейсов** с использованием **FastAPI**.

Для справки: Интернет (куча соединённых проводов), по своей сути, полагается на протокол управления передачей (**TCP** - Transmission Control Protocol) и интернет-протокол (**IP** - Internet Protocol) для обеспечения возможности передачи данных. Протокол **TCP** отвечает за разбиение данных на пакеты, обеспечение надежной доставки и повторную сборку их в пункте назначения. **IP**, с другой стороны, обрабатывает маршрутизацию этих пакетов по сетям, гарантируя, что они дойдут до предполагаемого получателя (грубо говоря, это адрес конкретной точки и способ их объединения в общую сеть).

Система доменных имен (**DNS** - Domain Name System) служит адресной книгой Интернета, переводя удобочитаемые доменные имена, такие как **"example.com"** на **IP-адреса** типа **"192.0.2.1"**. Этот процесс позволяет нам получать доступ к веб-сайтам без необходимости запоминать их конкретные **IP-адреса**.

В дополнение к **TCP**, **IP** и **DNS**, протокол передачи гипертекста (**HTTP** - Hypertext Transfer Protocol) является основополагающим для веб-коммуникации. HTTP позволяет веб-браузерам запрашивать ресурсы с веб-серверов, облегчая загрузку веб-сайтов, изображений, видео и другого контента. Через **HTTP** клиенты (такие как веб-браузеры) и серверы могут эффективно обмениваться информацией, обеспечивая динамичный веб-интерфейс (грубо говоря это структурированные правила общения клиентов с сервером, когда каждый понимает, что от него хотят).

Стоит также упомянуть про **JSON** (объектная нотация JavaScript - JavaScript Object Notation). Это легкий и удобочитаемый формат для структурирования данных, что делает его широко используемым в **веб-API** и передаче данных между серверами и клиентами. **JSON** основан на парах

ключ-значение, обеспечивая гибкий и эффективный способ представления сложных структур данных.

Если вдруг хотите поподробнее узнать про устройство интернета, HTTP и так далее, то мы бы рекомендовали пройти первую часть (прослушать лекции, не обязательно выполнять задания) курса **Web-технологии** (разбирают вглубь достаточно хорошо).

Если вам привычнее читать, то вот несколько ссылок:

- начало серии **статей** про устройство интернета (довольно техническое погружение);
- отдельно **статья** про HTTP;
- серия **статей** про интернет, но более доступно;
- **мега-гайд** (тоже может быть полезным).

FastAPI поддерживает различные методы **HTTP**, включая **GET**, **POST**, **PUT**, **DELETE** и другие, позволяя обрабатывать запросы разного типа. В зависимости от метода запроса сервер понимает, что именно хочет клиент: получить данные, отправить новую информацию, обновить существующую или удалить ресурс.

Чтобы обработать **HTTP-запрос**, достаточно определить функцию и украсить её соответствующим декоратором **FastAPI**.

Операции (Operations)

В контексте API **операция** — это выполнение конкретного **HTTP-запроса** к определённому пути.

FastAPI поддерживает все стандартные методы **HTTP**:

Основные:

- **GET** — получение данных (например, список пользователей).
- **POST** — создание нового ресурса (например, добавление нового пользователя).
- **PUT** — полное обновление ресурса (например, замена всех данных пользователя).
- **DELETE** — удаление ресурса.

Менее распространённые:

- **OPTIONS** — запрашивает, какие методы поддерживает сервер на этом URL.
- **HEAD** — запрашивает заголовки ответа без тела (похож на **GET**, но без данных).
- **PATCH** — частичное обновление ресурса (например, изменение только имени пользователя).
- **TRACE** — отладочный метод, возвращающий полученный запрос обратно клиенту.

Использование HTTP-операций в FastAPI

FastAPI делает работу с **HTTP-методами** максимально простой. Достаточно использовать соответствующий декоратор:

```
# main.py
from fastapi import FastAPI

app = FastAPI()
```

```
@app.get("/users")
def get_users():
    return {"message": "Список пользователей"}

@app.post("/users")
def create_user():
    return {"message": "Создан новый пользователь"}

@app.put("/users/{user_id}")
def update_user(user_id: int):
    return {"message": f"Пользователь {user_id} обновлён"}

@app.delete("/users/{user_id}")
def delete_user(user_id: int):
    return {"message": f"Пользователь {user_id} удалён"}
```

FastAPI также поддерживает экзотические методы:

```
# дополним main.py
@app.options("/")
def options_example():
    return {"message": "Этот запрос проверяет, какие методы доступны"}

@app.head("/")
def head_example():
    return {"message": "Ответ без тела (только заголовки)"}

@app.patch("/users/{user_id}")
def patch_user(user_id: int):
    return {"message": f"Пользователь {user_id} частично обновлён"}

@app.trace("/")
def trace_example():
    return {"message": "TRACE-запрос вернёт сам себя"}
```

Обработка данных форм с сайта

В дополнение к работе с **JSON**, **FastAPI** поддерживает обработку данных, отправляемых из **HTML-форм**. Это важно при интеграции **API** с веб-приложениями, которые используют классические формы для отправки данных.

Как FastAPI работает с формами: в **HTML-формах** данные отправляются в **формате application/x-www-form-urlencoded**. **FastAPI** позволяет обрабатывать такие запросы с помощью **Form**.

Пример обработки данных формы:

```
# main.py
from fastapi import FastAPI, Form

app = FastAPI()

@app.post("/submit/")
async def submit_form(
    username: str = Form(...),
    password: str = Form(...)
):
    return {"username": username, "password_length": len(password)}
```

Разбор кода:

- `@app.post("/submit/")` — этот эндпоинт принимает **POST-запросы**.
- `Form(...)` — указывает **FastAPI**, что данные параметра поступают из **HTML-формы**, а не **JSON**.
- **Пароль не передаётся в ответе** — в реальном приложении его нельзя показывать (здесь просто пример работы с данными).

Пример HTML-формы

Этот код создаёт простую форму, которая отправляет данные в наш эндпоинт:

```
<form action="http://127.0.0.1:8000/submit/" method="post">
  <input type="text" name="username" placeholder="Имя пользователя"
required>
  <input type="password" name="password" placeholder="Пароль" required>
  <button type="submit">Отправить</button>
</form>
```

Когда пользователь отправит форму, браузер выполнит **POST-запрос**, а **FastAPI** обработает данные.

Обработка нескольких полей

Можно передавать больше данных, например:

```
# дополним main.py
@app.post("/register/")
async def register_user(
    username: str = Form(...),
    email: str = Form(...),
    age: int = Form(...),
    password: str = Form(...)
):
    return {
        "username": username,
        "email": email,
```

```
    "age": age,
    "password_length": len(password)
}
```

А HTML-форма для него:

```
<form action="http://127.0.0.1:8000/register/" method="post">
  <input type="text" name="username" placeholder="Имя пользователя"
required>
  <input type="email" name="email" placeholder="Email" required>
  <input type="number" name="age" placeholder="Возраст" required>
  <input type="password" name="password" placeholder="Пароль" required>
  <button type="submit">Зарегистрироваться</button>
</form>
```

Когда использовать **Form**, а когда **Body**?

- **Form** подходит для обработки данных из **HTML-форм** (`application/x-www-form-urlencoded` или `multipart/form-data`).
- **Body** (`pydantic.BaseModel`) используется, когда данные приходят в формате **JSON** (`application/json`).

FastAPI позволяет легко работать с **HTML-формами**, автоматически валидируя и преобразовывая данные.

Работа с JSON в FastAPI

Мы ранее уже затрагивали работу с **JSON**, поэтому просто освежим наши знания. **FastAPI** использует модели **Pydantic** для автоматической проверки и анализа входящих данных запроса. Модели запросов помогают гарантировать, что входящие данные соответствуют указанной структуре и типам, в то время как модели ответов могут использоваться для определения структуры данных, возвращаемых вашим **API**.

FastAPI автоматически проверяет данные входящего запроса на соответствие определённым моделям **Pydantic**. Если данные не соответствуют указанной схеме, **FastAPI** возвращает ответ об ошибке с подробным объяснением ошибок проверки.

Пример модели Pydantic для проверки как входящего запроса, так и ответа:

```
# main.py
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class User(BaseModel):
    name: str
    age: int
```

```
@app.post("/users/")
async def create_user(user: User):
    return {"message": f"Пользователь {user.name} создан!", "user": user}
```

Разбор кода:

- `BaseModel` от **Pydantic** определяет структуру данных, которую ожидает **FastAPI**.
- Поля `name` и `age` автоматически проверяются на соответствие ожидаемым типам (`str` и `int`).
- Если клиент отправит, например, `age="двадцать"`, **FastAPI** вернёт ошибку валидации.

Работа с моделями ответа

FastAPI позволяет явно указывать модель, которую **API** должен использовать для формирования ответа:

```
# main.py
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class User(BaseModel):
    name: str
    age: int

class UserResponse(BaseModel):
    message: str
    user: User

@app.post("/users/", response_model=UserResponse)
async def create_user(user: User):
    return {"message": f"Пользователь {user.name} создан!", "user": user}
```

Здесь:

- `response_model=UserResponse` говорит **FastAPI**, что ответ должен соответствовать этой модели.
- Даже если внутри функции возвращается более сложная структура данных, **FastAPI** приведёт её к указанной модели.
- Мы применяем **вложенные модели Pydantic**, которые позволяют включать одну модель в другую для организации связанных данных. В нашем примере модель `User` используется внутри модели `UserResponse`, что упрощает валидацию и сериализацию вложенных структур в **JSON**. Обратите внимание, что вложенные модели могут применяться как при валидации входных, так и выходных данных.

Пример запроса:

```
{
  "name": "Артур",
  "age": 26
}
```

Пример ответа:

```
{
  "message": "Пользователь Артур создан!",
  "user": {
    "name": "Артур",
    "age": 26
  }
}
```

JSON-сериализация и автоматическое приведение типов

FastAPI и **Pydantic** автоматически конвертируют сложные типы данных в **JSON**. Например, если в модели есть `datetime`, он автоматически преобразуется в строку формата ISO 8601:

```
# main.py
from fastapi import FastAPI
from pydantic import BaseModel
from datetime import datetime

app = FastAPI()

class Event(BaseModel):
    name: str
    timestamp: datetime

@app.post("/events/")
async def create_event(event: Event):
    return event
```

При отправке:

```
{
  "name": "Уник",
  "timestamp": "2026-03-20T09:00:00"
}
```

Ответ тоже будет в этом формате. **FastAPI** сам разберёт строку `timestamp` и преобразует её в объект `datetime`.

Таким образом, использование моделей **Pydantic** делает работу с **JSON** в **FastAPI** безопасной и удобной, избавляя от необходимости вручную проверять и преобразовывать входные данные.

Обработка загрузки файлов

FastAPI поддерживает обработку загрузки файлов как части запроса. Вы можете использовать класс **File** из модуля **fastapi**, чтобы определить конечную точку загрузки файла и получить доступ к загруженным файлам в вашей функции роута.

Пример загрузки файлов:

```
# main.py
from typing import Annotated # про это будет чуть позднее в курсе
from fastapi import FastAPI, File, UploadFile

app = FastAPI()

@app.post("/files/")
async def create_file(file: Annotated[bytes, File()]):
    return {"file_size": len(file)}

@app.post("/uploadfile/")
async def create_upload_file(file: UploadFile):
    return {"filename": file.filename}
```

Разница между **File** и **UploadFile**

- **File**: загружает только содержимое файла в виде **bytes**, то есть в оперативную память. Хорошо подходит для небольших файлов.
- **UploadFile**: позволяет работать с файлами более эффективно, так как использует временные файлы, а не загружает всё в память. Также даёт доступ к метаданам файла, такой как имя, тип (**file.content_type**) и метод **file.read()**, позволяющий читать файл частями.

Обработка **UploadFile**

При использовании **UploadFile** можно работать с файлами построчно или читать их частями:

```
# пример обновления роута из main.py
@app.post("/uploadfile/")
async def create_upload_file(file: UploadFile):
    content = await file.read() # Читаем весь файл в память
    return {"filename": file.filename, "size": len(content)}
```

Если файлы могут быть большими, лучше не загружать их в память целиком. Вместо этого можно читать их порциями:


```
# другой пример обновления роута из main.py
@app.post("/uploadfile/")
async def create_upload_file(file: UploadFile):
    with open(file.filename, "wb") as f:
        while chunk := await file.read(1024): # Читаем по 1 КБ
            f.write(chunk)
    return {"filename": file.filename}
```

Загрузка нескольких файлов

FastAPI позволяет загружать сразу несколько файлов с помощью `List[UploadFile]`:

```
# main.py
from typing import List

# тут остальной код main.py, который дополним кодом ниже

@app.post("/multiple-files/")
async def upload_multiple_files(files: List[UploadFile]):
    return {"filenames": [file.filename for file in files]}
```

В HTML-форме это можно отправить так:

```
<form action="http://127.0.0.1:8000/multiple-files/" method="post"
  enctype="multipart/form-data">
  <input type="file" name="files" multiple>
  <button type="submit">Отправить</button>
</form>
```

Ограничение типов файлов

Чтобы ограничить загрузку определёнными типами файлов (например, только изображения), можно проверять `file.content_type`:

```
# новый роут в main.py
@app.post("/upload-image/")
async def upload_image(file: UploadFile):
    if file.content_type not in ["image/jpeg", "image/png"]:
        return {"error": "Только JPG и PNG разрешены"}
    return {"filename": file.filename, "content_type": file.content_type}
```

Более подробно можете прочитать тут: [Как загружать файлы в FastAPI](#)

Обработка параметров запроса

Параметры запроса являются неотъемлемой частью построения **API**. **FastAPI** предоставляет простой способ обработки параметров запроса, определяя их в качестве аргументов функции в ваших функциях маршрутизации.

Пример обработки параметров запроса:

```
# main.py
from fastapi import FastAPI

app = FastAPI()

fake_items_db = [{"item_name": "Foo"}, {"item_name": "Bar"}, {"item_name": "Baz"}]

@app.get("/items/")
async def read_item(skip: int = 0, limit: int = 10):
    return fake_items_db[skip : skip + limit]
```

Параметры запроса (Query Parameters)

Параметры запроса представляют собой набор пар ключ-значение, которые идут после знака **?** в **URL-адресе**, разделённые символами **&**.

Например, в этом **URL-адресе**:

```
http://127.0.0.1:8000/items/?skip=0&limit=10
```

Параметры запроса следующие:

- **skip**: со значением 0
- **limit**: со значением 10

FastAPI автоматически обрабатывает такие параметры и передаёт их в вашу функцию, как аргументы. В данном случае **skip** и **limit** являются параметрами запроса, и **FastAPI** будет автоматически преобразовывать их в типы данных, указанные в сигнатуре функции (в данном случае **int**).

Использование Query для настройки параметров запроса

В некоторых случаях вам может понадобиться задать дополнительные параметры для обработки параметров запроса, такие как ограничения по значениям или описание. Для этого вы можете использовать класс **Query** из модуля **fastapi**:

```
# main.py
from fastapi import FastAPI, Query

app = FastAPI()
```

```
@app.get("/items/")
async def read_item(
    skip: int = Query(0, alias="start", ge=0),
    limit: int = Query(10, le=100)
):
    return {"skip": skip, "limit": limit}
```

Пояснение:

- `alias="start"`: вы можете использовать другое имя для параметра запроса, например, `start` вместо `skip`.
- `ge=0`: задаёт ограничение на минимальное значение для `skip` (≥ 0).
- `le=100`: задаёт ограничение на максимальное значение для `limit` (≤ 100).

Пример запроса:

```
http://127.0.0.1:8000/items/?start=0&limit=10
```

Этот запрос эквивалентен:

```
http://127.0.0.1:8000/items/?skip=0&limit=10
```

Дополнительные возможности `Query`:

- `min_length` и `max_length`: для строковых параметров.
- `pattern`: для проверки строки на соответствие регулярному выражению.

Пример с проверкой строки с помощью `pattern`:

```
# main.py
from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/items/")
async def read_item(q: str = Query(..., pattern="^fixedprefix_")):
    return {"q": q}
```

Теперь параметр `q` должен начинаться с `fixedprefix_`.

Тело запроса и параметры пути

Помимо параметров запроса, **FastAPI** также поддерживает **параметры пути**, которые являются частями **URL-адреса**, используемого для идентификации ресурсов, о чем мы упоминали ранее

{**параметр**}). В дополнение можно отметить, что к параметрам пути можно добавить метаданные, задать им кастомный порядок, а также быстро провалидировать числовые данные (можете прочитать про импорт класса **Path** из **fastapi**).

Кроме того, вы можете передавать данные через **тело запроса**, используя (обычно) запросы **POST** и **PUT** для создания или обновления ресурсов.

Когда вам необходимо отправить данные из клиента (допустим, браузера, заполнив логин/пароль или отправив **HTML-форму**) в ваш **API**, вы отправляете их как **тело запроса**.

Тело запроса - это данные, отправляемые клиентом в ваш **API**. Тело ответа - это данные, которые ваш **API** отправляет клиенту. Чтобы объявить тело запроса, необходимо использовать **модели Pydantic, что вы уже умеете**.

Чтобы добавить параметр к вашему обработчику, объявите его также, как вы объявляли параметры пути или параметры запроса и укажите созданную модель в качестве типа параметра, в примере **Item**:

```
# main.py
from fastapi import FastAPI
from pydantic import BaseModel

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None

app = FastAPI()

@app.post("/items/")
async def create_item(item: Item):
    return item
```

<type> | <type> используется для объединения типов (создания более сложного типа). В данном случае мы говорим, что тип у **description** и **tax** может быть либо строкой/**float**, либо **None**, а также говорим, что параметр не обязателен (**= None**) - задаём значение по-умолчанию. Цепочку типов можно менять (но лучше быть аккуратным, чтобы не вываливалось **TypeError**), например **str | int | bool**.

Чтобы не было ошибок мы обычно используем только одно <или> вместе с **None**, и в коде проверяем на **None**, прежде чем выполнять действия с переменной. Так например у **None** нет метода **lower()**, поэтому если у нас строка **или None**, то мы сначала проверим на **None**, а только потом будем юзать методы строк.

Типы параметров и проверка

FastAPI предоставляет удобный синтаксис для проверки различных типов параметров, таких как `int`, `float`, `str`, `bool` и другие. Этим снимается значительная часть головной боли пайтон веб-разработчиков по ошибкам, которые возникают из-за некорректных типов входных данных, что тянуло за собой необходимость каждый раз держать в голове, что тип данных может не соответствовать ожидаемому.

Кроме того, вы можете использовать модели **Pydantic** для применения более сложных правил проверки и структур данных для параметров пути и запроса.

Объединение параметров пути и запроса

FastAPI позволяет использовать как параметры пути, так и параметры запроса в одном маршруте, что дает большую гибкость при проектировании **API-интерфейсов**. Это позволяет обрабатывать различные типы данных с одного эндпоинта, создавая **API**, который легко адаптируется под разные сценарии.

Пример с параметрами пути и запроса:

```
@app.get("/users/{user_id}")
def read_user(user_id: int, is_admin: bool = False):
    return {"user_id": user_id, "is_admin": is_admin}
```

В этом примере:

- Параметр пути `user_id` определяет, какой пользователь должен быть выбран.
- Параметр запроса `is_admin` указывает, является ли пользователь администратором, и имеет значение по умолчанию `False`, если не указан.

Различия между параметрами пути, запроса и тела запроса

1. Параметры пути (Path-параметры)

Параметры пути обычно используются для идентификации ресурса в **URL**. Они указываются в маршруте в фигурных скобках `{}`.

Пример:

```
@app.get("/{some_param}")
async def func_with_path_param(some_param: <type>):
```

Или с явным указанием:

```
from fastapi import Path

@app.get("/{some_param}")
async def func_with_path_param(some_param: Annotated[<type>, Path()]):
```

2. Параметры тела запроса (Body-параметры)

Для передачи данных в теле запроса чаще всего используют **Pydantic-модели**. Они указываются как тип данных в параметре функции маршрута.

Пример:

```
from pydantic import BaseModel

class User(BaseModel):
    name: str
    email: str

@app.post("/")
async def func_with_body_param(user: User):
```

Или с явным указанием:

```
from fastapi import Body

@app.post("/")
async def func_with_body_param(user: Annotated[User, Body()]):
```

3. Параметры запроса (Query-параметры)

Параметры запроса передаются в строке запроса (после знака **?**) и обычно используются для фильтрации или пагинации данных.

Пример:

```
@app.get("/")
async def func_with_query_param(query_param: <type>):
```

Или с явным указанием:

```
from fastapi import Query

@app.get("/")
async def func_with_query_param(query_param: Annotated[<type>, Query()]):
```

Заключение

- **Path-параметры** используются для динамической идентификации ресурса.
- **Query-параметры** передаются через строку запроса и могут использоваться для фильтрации или пагинации.

- **Body-параметры** передаются в теле запроса, обычно в формате **JSON**, и используются для создания или обновления данных.

Теперь у вас есть полное представление о том, как работать с параметрами пути и параметрами запроса в **FastAPI**. Эти инструменты позволяют строить гибкие и мощные **API-интерфейсы**, которые могут обрабатывать различные виды данных.

Made with ❤️ by **dvOretsky**